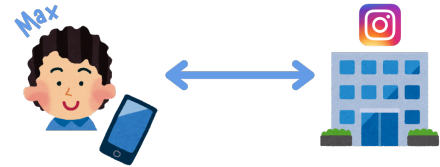


Passwörter

1. Login mit Account und Passwort

Um die Funktionen eines Passworts zu verstehen, betrachten wir folgende Leitfrage an einem konkreten Beispiel:

Woher weiss Instagram, dass sich wirklich Max mit seinem Konto anmelden möchte?



Stellen wir uns vor, Max möchte sich auf seinem Handy bei Instagram anmelden. Er gibt seinen Benutzernamen und sein Passwort ein. Instagram überprüft die Angaben und gewährt ihm anschliessend Zugriff auf sein Konto. Doch was passiert dabei eigentlich?

a) Identifikation

Zuerst muss Max angeben, wer er ist. Zum Beispiel gibt er seinen Benutzernamen oder seine E-Mail-Adresse ein. Das nennt man **Identifikation**. Die Identifikation beantwortet also die Frage: *Wer behauptest du zu sein?*

b) Authentifizierung

Nur zu behaupten, dass man Max ist, reicht natürlich nicht. Sonst könnte sich jeder als Max ausgeben. Instagram muss deshalb überprüfen, ob Max tatsächlich derjenige ist, für den er sich ausgibt. Daher gibt Max also sein Passwort an. Das nennt man **Authentifizierung**. Die Authentifizierung beantwortet also die Frage: *Kannst du beweisen, dass du wirklich Max bist?*

Ein Passwort ist dabei ein sogenannter **Authentifizierungsfaktor**. Es zur Kategorie «Etwas, das (nur) du weisst».

b) Autorisierung

Wenn Instagram erfolgreich überprüft hat, dass Max wirklich Max ist, stellt sich eine weitere Frage: *Was darf Max jetzt machen?*

Max darf beispielsweise seine eigenen Fotos ansehen, Nachrichten lesen oder Beiträge veröffentlichen. Er darf aber nicht einfach die privaten Nachrichten eines anderen Benutzers lesen oder dessen Konto verändern. Das nennt man **Autorisierung**. Die Autorisierung beantwortet also die Frage: *Was darfst du tun?*

Beispiele von Authentifizierungsfaktoren

Ein Passwort ist nur ein möglicher Authentifizierungsfaktor. Es gibt aber noch weitere Möglichkeiten, um die Authentifizierung durchzuführen. Dabei unterscheidet man zwischen 3 Kategorien:

Etwas, das du weisst

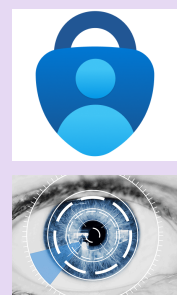
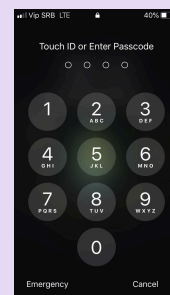
Dazu gehören zum Beispiel Passwörter, PINs, Sicherheitsfragen oder Muster.

Etwas, das du besitzt

Dazu gehören zum Beispiel Schlüssel, Smartphones oder Authenticator-Apps

Etwas, das du bist

Dazu gehören zum Beispiel ein Fingerabdruck, die Iris oder das Gesicht. Diese Kategorie wird auch als Biometrie bezeichnet.



Diese Faktoren können auch kombiniert werden. Wenn Max z.B. neben seinem Passwort noch einen Code aus einer Authenticator-App eingeben muss, verwendet er zwei Faktoren - auch **Multi-Faktor-Authentifizierung (MFA)** genannt. Das erhöht die Sicherheit, da ein Angreifer nun beide Faktoren kennen muss, um sich als Max auszugeben.

Jetzt haben wir geklärt, wie Instagram sich sicher sein kann, dass sich auch wirklich Max anmelden möchte. Nun stellt sich aber schon die nächste Frage: Wenn Instagram mein Passwort kennen muss, muss Instagram es dann auch irgendwo speichern?

2. Wie werden Passwörter gespeichert?

Vorhin haben wir gesehen, dass Max sich bei Instagram mit seinem Benutzernamen und Passwort einloggen muss. Nehmen wir mal an, Max verwendet das Passwort «Sommer2026!». Eine sehr einfache Möglichkeit wäre, dass Instagram dieses Passwort direkt in seiner Datenbank speichert.

Die Datenbank könnte beispielsweise so aussehen wie in Tabelle 1. Das nennt man **Klartextspeicherung**. Wenn Max sich anmeldet, kann Instagram das eingegebene Passwort einfach mit dem gespeicherten Passwort vergleichen. Das funktioniert zwar technisch, ist aber äusserst unsicher... Warum?

Benutzer	Passwort
Max	Sommer2026!
Anna	Fussball123
Luca	Mauzi
Sara	123456789

Tabelle 1: Datenbank mit Klartext

Die Übermittlung selbst kann man zwar mit einer Verschlüsselung irgendwie sichern, sodass keine Eve das Passwort abfangen kann. Aber in dieser Situation gibt es noch eine weitere Schwachstelle. Angenommen, ein Angreifer verschafft sich Zugriff auf die Datenbank bei Instagram. Dann könnte er sofort alle (!) Passwörter lesen! Er müsste die Passwörter also nicht einmal knacken, sondern könnte sie einfach so verwenden. Deshalb sollten Passwörter niemals im Klartext gespeichert werden.

2.1. Die Idee der Hashfunktion

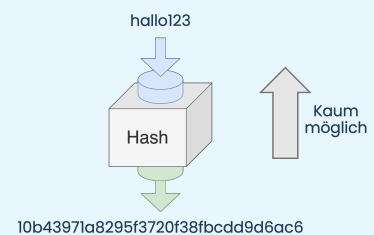
Hier kommt ein Prinzip ins Spiel, das wir schon aus der Kryptologie bei der asymmetrischen Verschlüsselung kennengelernt haben: Die Hashfunktion.

Repetition Hashfunktion

Wir haben schon einmal die Hashfunktion kennengelernt. Sie hat die Eigenschaft, dass sie eine Input nimmt und in eine ganz andere Form bringt. Dabei ist es praktisch unmöglich, aus dem Hashwert wieder das ursprüngliche Passwort zu berechnen (Beispiel Spaghetti kochen).

Ein weiteres Merkmal in der Informatik bei Hashfunktionen ist, dass sie eine beliebige Länge als Input reinnimmt (z.B. ein Passwort) und daraus einen Hashwert mit einer fixen Länge erstellt. Das heisst, egal wie lang das Passwort ist, die gespeicherten Hashwerte haben immer die gleiche Länge.

Der konkrete Hashwert hängt natürlich von der verwendeten Hashfunktion ab. Wichtig dabei ist: Aus demselben Passwort entsteht bei derselben Hashfunktion immer derselbe Hashwert.



Das Ganze sieht einer Verschlüsselung verdächtig ähnlich... Aber hier müssen wir eine Unterscheidung machen:

Bei einer Verschlüsselung wird eine Nachricht so verändert, dass sie ohne den passenden Schlüssel nicht gelesen werden kann.

Beim Hashing wird aus einer Eingabe ein Hashwert berechnet. Und dieser Wert ist nicht dazu gedacht, wieder in die ursprüngliche Eingabe zurückverwandelt zu werden. Deshalb spricht man hier von einer Einwegfunktion.

insert image vergleich verschlüsselung hash

Wie kann Instagram dann das Passwort überprüfen?

Bei der Registrierung von Max speichert Instagram nicht sein Passwort als Klartext, sondern übersetzt dieses mit einer Hashfunktion in einen Hashwert (vereinfacht auch Hash genannt). Die Datenbank sieht also nicht mehr wie in Tabelle 1 aus, sondern wie in Tabelle 2. Das eigentliche Passwort ist nicht mehr ersichtlich.

Benutzer	Passwort
Max	0ec22f12d2aef35308467e125d3996de
Anna	7ed70c52ce29499ba647a775cec40d6d
Luca	2981b5bf5f6772a8b0f2c43b5f98e80a
Sara	25f9e794323b453885f5181f1b624d0b

Tabelle 2: Die Datenbank bei Instagram speichert nicht das Passwort als Klartext, sondern den Hashwert des Passworts.

Einige Tage später möchte Max sich wieder anmelden. Instagram wird nun nicht das Passwort direkt mit dem gespeicherten Passwort vergleichen, sondern es zuerst wieder mit derselben Hashfunktion in einen Hashwert umwandeln. Dann werden die zwei Werte verglichen. Wenn sie übereinstimmen, wird die Anmeldung akzeptiert, andernfalls abgelehnt.

Der Server bei Instagram muss das eigentliche Passwort also nicht kennen, um überprüfen zu können, ob das richtige Passwort eingegeben wurde.

2.2. Besonderheiten bei Hashes

Nun ergeben sich natürlich einige Besonderheiten bezüglich dieses Verfahrens. Gehen wir nun auf diese ein.

Avalanche Effekt

Eine wichtige Grundvoraussetzung ist, dass man nicht aus dem Hashwert das Passwort irgendwie zurückrechnen kann. Daher ist es auch wichtig, dass sich der Hashwert bei einer kleinen Änderung des Passworts stark verändert.

Im Beispiel im Bild sieht man, dass eine Änderung eines einzelnen Zeichens im Passwort dazu führt, dass der Hashwert komplett anders aussieht. Das nennt man den **Avalanche Effekt** oder **Streuung**.

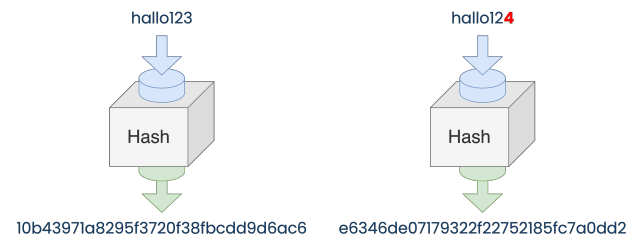


Abbildung 2: Nur ein einziges Zeichen verändert den Hashwert ganz.

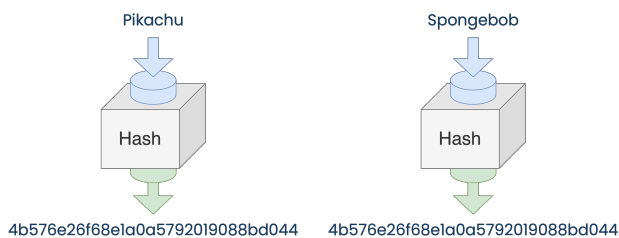


Abbildung 3: Verschiedene Passwörter ergeben denselben Hash.

Kollisionen

Wir haben bereits gesehen, dass Hashfunktionen eine Eingabe in einen Hashwert fester Länge umwandeln. Damit entsteht ein interessantes mathematisches Problem: Es gibt sehr viele mögliche Passwörter, aber nur eine begrenzte Anzahl möglicher Hashwerte. Deshalb ist es theoretisch möglich, dass zwei verschiedene Eingaben denselben Hashwert erzeugen. Das nennt man eine **Kollision**. Bei einer guten kryptografischen Hashfunktion soll es allerdings extrem schwierig sein, absichtlich zwei passende Eingaben mit demselben Hash zu finden.

Bekannte Hashfunktionen

2.3. Rainbow Tables und Salting

So, jetzt stellt sich die Frage, ob ein Angreifer, der Zugriff auf die Datenbank hat, die Passwörter trotzdem irgendwie herausfinden kann.

Aufgabe 1

Überlegen Sie sich, wie ein Angreifer vorgehen könnte, um die Passwörter zu knacken, falls er so eine Tabelle wie in Tabelle 2 gestohlen hat und nun besitzt.

Falls nun also ein Angreifer in den Besitz einer solchen Benutzernamen-Hash-Tabelle (wie in Tabelle 2) kommt, könnte er eventuell durch Ausprobieren beliebiger Passwörter die Passwörter der Benutzer herausfinden.

Sara's Passwort in Tabelle 1 ist zum Beispiel «123456789» gewesen. Falls der Angreifer nun selbst auf die Idee kommt, dass viele Leute dieses Passwort verwenden könnten, könnte er dieses Passwort selbst hashen und mit den Hashwerten in der Tabelle vergleichen. Er würde dann feststellen, dass der Hashwert übereinstimmt und somit wüsste er mit Sicherheit, dass Sara's Passwort «123456789» ist.

Das nennt man einen **Offline-Angriff**, weil der Angreifer kann die gestohlenen Daten in Ruhe auf seinem eigenen System untersuchen, ohne bei Instagram für jeden Versuch einen Login durchführen zu müssen.

Beliebte Passwörter (ein paar Beispiele)

- password • 123456
- qwertz • abc123
- iloveyou • 123123

Rainbow Tables

Eine Rainbow Table ist vereinfacht gesagt eine vorberechnete Sammlung von Passwort- und Hashwerten. Quasi wie ein Nachschlagewerk für Angreifer.

Wenn ein Angreifer einen gestohlenen Hash besitzt, kann er versuchen, den passenden Eintrag zu finden. Solche vorbereiteten Tabellen sind also besonders für schwache Passwörter eine gute Angriffsmöglichkeit.

Aufgabe 2

Sie sind eine bekannte oder ein bekannter AngreiferIn und haben durch einen internen Maulwurf bei Instagram den Zugriff auf eine Tabelle mit Benutzernamen und deren gehashed Passwörtern bekommen:

Benutzer	Passwort	Benutzer	Passwort
Max	3d46feca7646fe8451af309a9f014123	Nina	6049e0b0903501a92fffb52ea35ff91f
Anna	a864a7d0f33a71467c81e7724df0020d	Tim	3d54cf48eb2dee70ccf8a1e40c621d03
Luca	7ed70c52ce29499ba647a775cec40d6d	Lea	c4aaa2e7db25e4042eda12c47668d341
Sara	3d6cfefb3fefefb2ae3310444ad66d13b	Jonas	e2b764068994715ed3bc13c21ac3ad79

Praktischerweise haben Sie auch schon Ihre Rainbow Table bereit:

Passwort	Hash	Passwort	Hash
Pizza123	c4aaa2e7db25e4042eda12c47668d341	Fussball	3d54cf48eb2dee70ccf8a1e40c621d03
Mauzi454	3d6cfefb3fefefb2ae3310444ad66d13b	Apfel123	f1c505746bdcd8e8d9b28513fd0a591
Passwort1	e2b764068994715ed3bc13c21ac3ad79	CoffeeLover	f943ef55cd7654f184274cb8f6d7422b
Sommer0	3d46feca7646fe8451af309a9f014123	Sonne	6049e0b0903501a92fffb52ea35ff91f
Hallo123	c10fedc1a97741855818849f936d2463	Test1234	8a8c05746bdcd8e8d9b28513fd0a591

Bei wie vielen Benutzern aus der ersten Tabelle kann das Passwort mithilfe der Rainbow-Tabelle herausgefunden werden? Gib die entsprechenden Benutzernamen und Passwörter an.

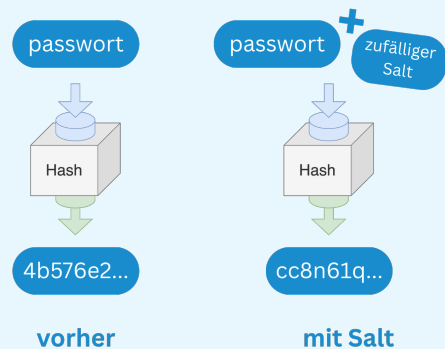
Da überall in diesen Firmen auch nur Menschen (potentielle Schwachstellen...) arbeiten, ist es gar nicht so unwahrscheinlich, dass Unbefugte an solche Benutzernamen-Hash-Tabellen rankommen. Daher müssen wir die Sicherheit noch etwas mehr aufdrehen. Hier kommt der Salt ins Spiel:

Salting

Ein Salt ist eine zufällig erzeugte Zeichenfolge, die vor dem Hashen mit dem Passwort kombiniert wird. Der Salt ist kein Geheimnis. Er darf zusammen mit dem Hash gespeichert werden.

Warum hilft der Salt?

Nehmen wir mal an, dass Max und Nina dasselbe Passwort verwenden. Ohne Salt hätten beide denselben Hash und der Angreifer könnte auf einen Schlag von 2 Benutzern das Passwort erkennen. Mit dem Salt entstehen unterschiedliche Hashwerte und für den Angreifer wird es so viel schwieriger, eine einzige vorberechnete Tabelle für alle Benutzer zu verwenden.



3. Angriffe

Wir haben nun gesehen, wie Passwörter gespeichert und bei einem Login überprüft werden. Im Kapitel vorhin haben wir schon ein paar Mal erwähnt, wie Angreifer vorgehen können. Aber (leider) gibt es eine Vielzahl an Methoden. Wir unterscheiden deshalb verschiedene Arten von Angriffen und werden ein paar in diesem Kapitel thematisieren.



3.1. Brute Force

Beginnen wir mit der einfachsten Idee: Ein Angreifer probiert möglichst viele mögliche Passwörter aus. Dieses Vorgehen nennt man **Brute Force** (dt.: rohe Gewalt).

Stellen wir uns vor, Nina verwendet eine vierstellige PIN, um ihr Handy zu sperren. Ein Angreifer könnte hier systematisch ausprobieren und alle Möglichkeiten (Brute Force) durchgehen: 0000, 0001, 0002, 0003, ..., 9999 bis er irgendwann auf die richtige Kombination kommt.

Bei 4 Ziffern gibt es insgesamt $10^4 = 10'000$ Möglichkeiten. Hört sich zwar nach viel an, ein Computer kann solche Verfahren aber sehr schnell durchführen.

Die Anzahl der möglichen Passwörter hängt also unter anderem von der Länge und von der Anzahl möglicher Zeichen ab. Wenn ein Passwort aus 26 Kleinbuchstaben besteht, gibt es bei:

- 1 Zeichen $\rightarrow$ 26 Möglichkeiten
- 2 Zeichen $\rightarrow 26^2 = 676$ Möglichkeiten
- 8 Zeichen $\rightarrow 26^8 = 208'827'064'576$ Möglichkeiten

Allgemein gilt: Anzahl Möglichkeiten = Anzahl möglicher Zeichen^{Passwortlänge}. Man erkennt dadurch schnell, dass jedes zusätzliche Zeichen den Suchraum massiv vergrößert.



Aufgabe 3

Berechne, wie viele verschiedene Kombinationen möglich sind.

Passwort	Mögliche Zeichen	Länge	# Kombinationen
abcdf	Kleinbuchstaben	4	?
123abc	Kleinbuchstaben + Ziffern	6	?
Informatik	Klein- und Grossbuchstaben	10	?
H3ll0	Klein- und Grossbuchstaben + Ziffern	5	?

Bei Brute-Force kann man zwischen zwei Situationen noch weiter unterscheiden: Online und offline Brute Force Angriffe:

Online

Der Angreifer versucht sich direkt beim Dienst anzumelden. Er versucht also einen 1. Login-Versuch z.B. bei Instagram mit Ihrem Benutzernamen. Vermutlich wird dann «Passwort falsch» erscheinen. Also geht der Angreifer weiter zum nächsten Login-Versuch usw.

Der Dienst kann die Attacke erschweren, indem er:

- die Anzahl der Loginversuche begrenzt,
- zwischen Versuchen wartet,
- verdächtige Anmeldungen blockiert,
- ein Captcha verlangt.

Wichtiger Unterschied

Bei einem Online-Angriff muss der Server jeden Versuch bearbeiten. Bei einem Offline-Angriff kann der Angreifer die gestohlenen Daten selbst untersuchen.

Offline

Anders sieht es aus, wenn der Angreifer eine Datenbank mit Passwort-Hashes gestohlen hat. Dann muss er nicht mehr bei Instagram jedes Mal nachfragen, sondern kann auf seinem Computer rechnen (haben wir weiter oben schon mal gesehen). Er stiehlt also den Hash. Dann probiert er ein Passwort am eigenen Computer aus, berechnet davon den Hash und vergleicht es mit dem gestohlenen Wert.

3.2. Wörterbuchattacken

Brute Force hat allerdings ein Problem: Menschen wählen ihre Passwörter nicht zufällig. Es wäre sehr ineffizient, zuerst Millionen völlig zufällige Kombinationen auszuprobieren, wenn viele Menschen klassische Passwörter wählen, wie wir schon weiter oben gesehen haben: *password*, *1234*, *qwertz*, ...

Ein Angreifer kann deshalb eine Liste mit häufig verwendeten Passwörtern verwenden. Das nennt man eine **Wörterbuch-attacke**. Dabei wird nicht unbedingt nur ein normales Wörterbuch verwendet, sondern diese Listen beinhalten auch:

- häufig verwendeten Passwörtern
- Namen
- Orten
- häufigen Zahlenfolgen
- bekannten Passwortmustern

Zusätzlich können Varianten ausprobiert werden: *sommer*, *Sommer*, *Sommer1*, *Sommer!*, *Sommer2026*, ... Der Angreifer versucht also, möglichst gut vorherzusagen, welches Passwort ein Mensch gewählt haben könnte.

Öffentlich zugängliche Informationen, entweder über Webseiten oder Social Media Profilen, können diese Attacken noch zusätzlich unterstützen. Wenn Max also auf seinem öffentlichen Instagram-Profil Informationen über seinen Geburtstag, Lieblingsverein, Hund und seine Freundin teilt, kann ein Angreifer diese Informationen benutzen, um das Wörterbuch geschickt zu erweitern:



Abbildung 4: Max hat auf Social Media viele Informationen über sich geteilt.

- Max*19032012
- YBbesteVerein
- Bello123

Aufgabe 4

Ordne die folgenden Angriffe richtig zu: Brute Force oder Wörterbuch-Attacke?

Versuch	Brute Force	Wörterbuch-Attacke
000000, 000001, 000002, ...	☐	☐
password, hallo, qwertz, ...	☐	☐
Sommer2026!, Sommer2027!, ...	☐	☐
a, b, c, d, e, ...	☐	☐

3.3. Rainbow Tables

Haben wir vorher schon in Abschnitt 2 gesehen.

3.4. Credential Stuffing

Bisher haben wir betrachtet, wie ein Angreifer ein Passwort herausfinden kann. Aber manchmal muss er das gar nicht. Stellen wir uns vor, eine Webseite wird gehackt. Das ist übrigens gar nicht so unwahrscheinlich :(

Entweder hat der Angreifer selbst die Seite gehackt und somit von allen Benutzern direkt den Namen und Passwort oder er kauft solche gestohlenen Daten im Darknet. Der Angreifer weiss nun, dass z.B. Max auf der Webseite das Passwort «Sommer2026!» verwendet.

Vielleicht verwendet Max dasselbe Passwort auch bei anderen Diensten? Beim **Credential Stuffing** versucht der Angreifer nun einfach bei weiteren Diensten mit demselben Email/Benutzernamen und Passwort die Konten von Max zu hacken. Zum Beispiel bei Instagram, Discord, Gmail, usw. Mit einem Computer und automatisierten Programm kann man auf tausenden von Seiten extrem schnell diese Login-Versuche durchführen.

3.5. Phishing

3.6. Was passiert bei einem Datenleck?

4. Zentrales Login: Unser Email

5. Bad & Good Practices

Lösungen

Lösung 1

Der Angreifer könnte eine Liste von möglichen/beliebten Passwörtern nehmen und diese mit derselben Hashfunktion in Hashwerte umwandeln. Dann könnte er die berechneten Hashwerte mit den gestohlenen Hashwerten vergleichen. Wenn er eine Übereinstimmung findet, weiss er, welches Passwort zu welchem Benutzer gehört.

Lösung 2

- Max mit Sommer0
- Sara mit Mauzi454
- Tim mit Fussball

Lösung 3

Lösung 4